

Inaccurate Results in TSP Problem Solution with tsp.py and QAOA

<https://github.com/qiskit-community/qiskit-optimization/issues/511>

ROW 56

New issue



- Qiskit Optimization version: 0.4.0
- Python version: 3.10.7
- Operating system: Windows 11 Home

I encountered difficulties while attempting to implement the Traveling Salesman Problem (TSP) using the QAOA function from Qiskit Optimization, as demonstrated in [1]. Unfortunately, the results were not satisfactory compared to the VQE approach. I also utilized the `tsp.py` function from [3] as provided in [1]. While I managed to obtain the correct energy value, I encountered issues with the sampling and evaluation of the values, leading to inaccurate results. This situation has made it challenging to solve the remaining aspects of the problem.

- [1] TSP Implementation: [Link](#)
- [2] Qiskit QAOA Function: [Link](#)
- [3] TSP.py Function: [Link](#)

```

# Create a random TSP Hamiltonian
qp = tsp.to_quadratic_program()
print(qp.prettyprint())

qp2qubo = QuadraticProgramToQubo()
qubo = qp2qubo.convert(qp)
qubit0p, offset = qubo.to_ising()
print(qubit0p, offset)

algorithm_globals.random_seed = 10598

optimizer = COBYLA()
sampler = Sampler()
qaoa = QAOA(sampler, optimizer, reps=2)

# solving the Problem using QAOA

qaoa_result = qaoa.compute_minimum_eigenvalue(qubit0p)

print("\nQAOA:\n", qaoa_result)

#u = qaoa_result.optimal_circuit

lowest_eigen_value = qaoa_result.best_measurement['value']
print("energy:", lowest_eigen_value)

print("tsp objective:", lowest_eigen_value.real + offset)
print("time:", result.optimizer_time)
sample = tsp.sample_most_likely(qaoa_result.eigenstate)
print("sample:", sample)
print(tsp.edgelist(sample))
z = tsp.interpret(sample)

#print('tsp_draw_result(qaoa_result)',tsp_draw_result[samp
print("solution:", z)
print("solution objective:", tsp.tsp_value(z, adj_matrix))

```

```
QA0A:
SamplingMinimumEigensolverResult:
  Eigenvalue: -1079.925858180594
  Best measurement
  : {'state': 84, 'bitstring': '001010100', 'value': (-7379+0j), 'probability': 0.0067433079007692}

tsp objective: 6501.074141819496
energy: -216.54100224461524
time: 4.109718561172485
feasible: True
solution: [0, 0, 0]
solution objective: 0.0
```

No one assigned

No labels

No type

No projects

No milestone

None yet

No branches or pull requests

You're not receiving notifications from this thread.

VQE Code and Output:

```
from qiskit.circuit.library import TwoLocal
from qiskit.algorithms.optimizers import SPSA
from qiskit.algorithms.minimum_eigensolvers import SamplingVQE

algorithm_globals.random_seed = 123
seed = 10598
optimizer = SPSA(maxiter=300)
ry = TwoLocal(qubit0p.num_qubits, "ry", "cz", reps=5, entanglement="linear")
vqe = SamplingVQE(sampler=Sampler(), ansatz=ry, optimizer=optimizer)

result = vqe.compute_minimum_eigenvalue(qubit0p)

print("energy:", result.eigenvalue.real)
print("time:", result.optimizer_time)
x = tsp.sample_most_likely(result.eigenstate)
print("feasible:", qubo.is_feasible(x))
z = tsp.interpret(x)
print("solution:", z)
print("solution objective:", tsp.tsp_value(z, adj_matrix))

draw_tsp_solution(tsp.graph, z, colors, pos)

energy: -7326.02469952184
time: 22.964696168899536
feasible: True
solution: [1, 2, 0]
solution objective: 202.0
```

How can we reproduce the issue?

Revisiting the source code for both QAOA and the TSP implementation would help identify any incompatibilities or discrepancies between them.

What should happen?

It is expected that the energies obtained from VQE and QAOA should match for small graphs in the context of the Traveling Salesman Problem (TSP). Additionally, the TSP functions utilized in QAOA should function similarly to those used in VQE. However, in my implementation, I have observed discrepancies between the results of VQE and QAOA for the energy values.

Any suggestions?

If there are any mistakes or areas where I may have gone wrong, I kindly request your guidance and assistance. I am open to any suggestions or advice to help resolve the issues I encountered during the implementation. Your expertise and insights would be greatly appreciated.



Syed-Affan-Hussain changed the title ~~solving TSP with tsp.py and QAOA is giving incorrect answers~~ Inaccurate Results in TSP Problem Solution with `tsp.py` and QAOA on May 13, 2023



t-imamichi on May 15, 2023 · edited by t-imamichi

Edits ▾ Collaborator ...

You need to try larger `maxiter` and `reps` for QAOA. VQE and QAOA has no guarantee that the final solution converges to the optimal solution. So, your situation can happen, especially with small `maxiter` and `reps`.



Syed-Affan-Hussain on May 15, 2023

Author ...

Thank you so much I set maxiter to 3000 and rep to 1, surprisingly it worked



 **Syed-Affan-Hussain** changed the title ~~solving TSP with tsp.py and QAOA is giving incorrect answers~~ **Inaccurate Results in TSP Problem Solution with tsp.py and QAOA** on May 13, 2023



t-imamichi on May 15, 2023 · edited by t-imamichi

Edits ▾ Collaborator ...

You need to try larger `maxiter` and `reps` for QAOA. VQE and QAOA has no guarantee that the final solution converges to the optimal solution. So, your situation can happen, especially with small `maxiter` and `reps`.



Syed-Affan-Hussain on May 15, 2023

Author ...

Thank you so much I set `maxiter` to 3000 and `rep` to 1, surprisingly it worked



woodsp-ibm on May 15, 2023

Member ...

I will also note that QAOA is simply a subclass of SamplingVQE where QAOA fixes the ansatz to use the QAOAAnsatz. To compare more adequately choosing the same optimizer and `algorithm_globals` seed would minimize other differences. In QAOA you use COBYLA and in VQE the SPSA. The seeds are also different which affects the start point, which in the absence of being set explicitly its chosen randomly. Of course the start point will mean different things to the ansatzes (different starting state result) which may affect the outcome and make the direct comparison more tricky.

With `Sampler()` from `qiskit.primitives`, by default, as you created it above it gives the ideal distribution - created thus it has no sampling/shot noise. SPSA was designed to work in noisy environments and works rather differently that say the COBYLA. Given the above has an ideal outcome you could use SLSQP, a classical gradient based optimizer. Either way choosing the same optimizer would reduce the differences for the comparison.

In the end the real difference would/should come down to just the ansatz.



 **t-imamichi** closed this as completed on May 16, 2023